INSA INSTITUT NATIONAL DES SCIENCES APPLIQUÉES ROUEN NORMANDIE

Département ITI

Projet intégratif : Smart Robot

Rapport de projet
Smart Robot

Étudiants

Célia	ANTUNES
Edouard	DOUBET
Adam	EL MARZOUKI
Bréhan	ERAUD-ROBERT
Hadi	ASSI
Romain	LEPREVOST

14 octobre 2026

Table des matières

Introduction **1**

1 Présentation du sujet **2**
1.1 Description générale du problème . . . 2
1.2 Format du fichier de description de la ville . . . 2
1.3 Différents ordres pour le robot . . . 2

2 Partie électronique **3**
2.1 Schéma électrique . . . 3
2.2 Architecture matérielle du robot . . . 4
2.2.1 Raspberry Pi : cœur du système . . . 4
2.2.2 Capteurs de ligne TCRT5000 (x5) . . . 4
2.2.3 L293D : driver de moteurs . . . 5
2.2.4 Moteurs DC . . . 5
2.2.5 Alimentation des moteurs (pile 9V) . . . 6
2.2.6 Capteur de couleur TCS34725 . . . 6
2.2.7 Écran LCD 16x2 avec interface I2C . . . 7
2.2.8 Buzzer actif . . . 7
2.3 Choix techniques et câblage . . . 7
2.4 Bilan de puissance et estimation de l'autonomie . . . 9
2.4.1 Circuit Logique (Batterie Externe USB) . . . 9
2.4.2 Circuit de Puissance (Cellules Lithium) . . . 10
2.4.3 Conclusion . . . 10
2.5 Difficultés rencontrées et pistes d'amélioration . . . 10
2.6 Flowchart . . . 11

3 Partie informatique **12**
3.0.1 Analyse . . . 12
3.0.2 Conception Préliminaire . . . 15
3.1 Conception détaillée . . . 18
3.1.1 Fonctions du Labyrinthe . . . 18
3.1.2 Fonctions du Robot . . . 18
3.1.3 Algorithme générale . . . 20
3.2 Implémentation en C . . . 24
3.2.1 Organisation des fichiers . . . 24
3.2.2 Tests unitaires . . . 26
3.2.3 Documentation . . . 33

4 Démonstration **34**
4.1 Procédure de lancement 34
4.2 Mesure du temps 34
4.3 Comportement observé 34
4.3.1 Première tentative 34
4.3.2 Deuxième tentative 35
4.3.3 Troisième tentative 35

Conclusion **36**
4.4 Utilisation de l'IA / de LLM 37

Introduction

Dans le cadre de ce projet intégratif, nous étions chargés de concevoir un robot capable de répondre à une problématique imposée. Ce projet avait pour objectif de mettre en œuvre les différentes compétences acquises lors du cours d'Algorithmique ainsi que celui d'Électronique pour les systèmes embarqués. La gestion de projet était également importante pour assurer le suivi et la coordination des différentes étapes du projet.

Ce projet se divise en deux parties complémentaires. La première partie est une partie algorithmique, qui consiste en l'analyse de la description de la ville. Afin de déterminer le parcours le plus optimal, et préparer une liste d'instructions pour le robot. La deuxième partie est une partie robotique, qui consiste à concevoir le robot, et faire un programme capable de le piloter, et d'exécuter les différentes instructions générées lors de la première partie.

Le cahier des charges pour le projet sera le suivant :
— Le robot doit être capable de suivre des lignes noires et de corriger sa trajectoire en cas de déviations.
— Le robot doit être capable de franchir un certain nombre de points de passage imposés.
— Le robot doit être capable de revenir à son point de départ.
— L'algorithme doit être capable de trouver le parcours le plus court possible.

Le rapport sera donc structuré de la manière suivante. Dans une première partie, nous présenterons, de manière plus détaillée les contraintes techniques du sujet, l'organisation du projet et les différentes étapes de conception. Dans une seconde partie, nous présenterons la conception électronique du robot, incluant le schéma électrique, les choix techniques et le bilan de puissance du robot. Ensuite, nous aborderons dans la troisième partie, les choix de conception et d'implémentation de notre robot. Celui-ci suivra le cycle en V, et abordera les algorithmes clés. Dans un dernier temps, nous ferons un bilan de la démonstration, avec un commentaire sur les performances du robot, et d'éventuels problèmes rencontrés. Enfin, nous ferons une synthèse du projet, en abordant les différents problèmes rencontrés et les solutions proposées à une échelle plus globale.

Chapitre 1

Présentation du sujet

1.1 Description générale du problème

Dans cette partie, nous allons passer en revue les différentes contraintes imposées, et l'organisation générale de notre groupe dans le cadre du projet. Le but n'est pas d'aborder en détail ce que nous avons réalisé, mais plutôt de passer en revue le déroulement général du projet.

1.2 Format du fichier de description de la ville

L'entrée pour notre programme sera un fichier texte décomposé comme suit :
— Première ligne : largeur l (nombre de cases par côté).
— Deuxième ligne : case de départ + orientation (par exemple `6S`).
— Lignes suivantes : passages entre cases (`1-2`, `1-6`, etc.).
— Dernières lignes : numéros des cases contenant les points de passage.
— Fin du fichier : un point `.`.

Ce fichier sera lu par notre programme pour comprendre la ville et les points de passage, une fois le programme terminé, il renverra une liste d'instructions pour le robot. Regardons donc un peu plus en détail quels sont les ordres que le robot peut recevoir.

1.3 Différents ordres pour le robot

— `AV` : avancer jusqu'au prochain croisement ;
— `TG` : tourner à gauche (sans avancer) ;
— `TD` : tourner à droite (sans avancer).

Ces ordres seront lus par notre programme et exécutés par le robot pour réaliser le parcours complet. Le programme de création des instructions lit un fichier, spécifié en paramètre de la commande, afin d'obtenir les informations de la ville. Son exécution produit une sortie dans la sortie standard, que nous récupérons dans l'entrée standard du programme chargé de déplacer le robot. L'exécution des programmes ressemble donc à `./algo/bin/laby ville.txt | ./elec/bin/robot`.

Chapitre 2

Partie électronique

2.1 Schéma électrique

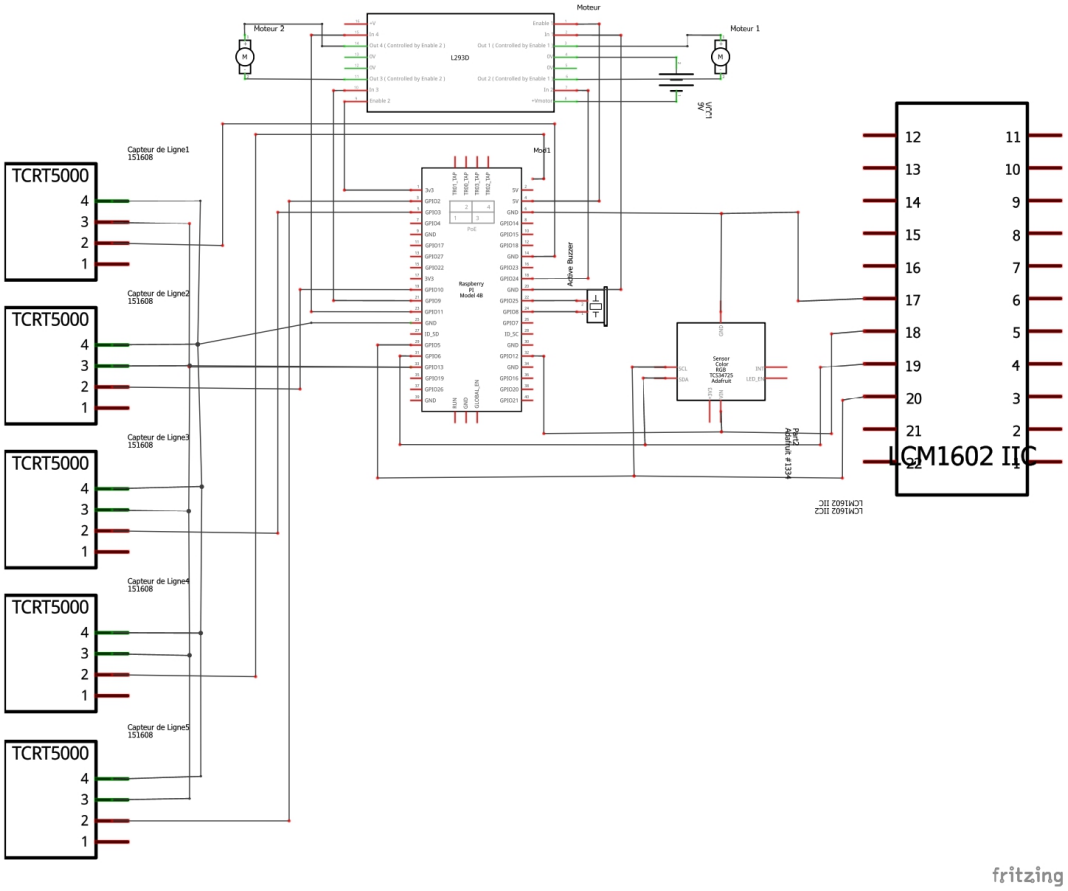


FIGURE 2.1 – Schéma électrique complet du robot.

Le module I2C, sur la boîte est normalement branché à l'écran LCD, mais il n'est pas branché par lisibilité du schéma, et que notre module était déjà branché de notre côté.

2.2 Architecture matérielle du robot

2.2.1 Raspberry Pi : cœur du système

Rôle

Le Raspberry Pi constitue le cerveau du robot. Il assure les fonctions suivantes :
— lecture des données provenant des capteurs,
— traitement des informations,
— commande des moteurs,
— communication avec l'écran LCD et le capteur de couleur.

Connexions principales

— GPIO → capteurs de ligne TCRT5000,
— GPIO → driver de moteurs L293D,
— GPIO I2C (SDA, SCL) → écran LCD et capteur de couleur,
— GPIO → buzzer,
— Alimentation 5V et GND → ensemble des modules.

2.2.2 Capteurs de ligne TCRT5000 (x5)

Rôle

Les capteurs TCRT5000 sont des capteurs infrarouges utilisés pour :
— détecter une ligne noire sur un fond clair (suivi de ligne),
— déterminer la position de la ligne sous le robot.
Chaque capteur est composé :
— d'une LED infrarouge émettrice,
— d'un phototransistor récepteur.

Connexions

Chaque capteur TCRT5000 est alimenté par le Raspberry Pi et connecté à un GPIO distinct afin de permettre une lecture indépendante de la position de la ligne.
— **Capteur arrière gauche :**
  — VCC → 3.3V du Raspberry Pi,
  — GND → GND du Raspberry Pi,
  — OUT → GPIO21.
— **Capteur avant gauche :**
  — VCC → 3.3V du Raspberry Pi,
  — GND → GND du Raspberry Pi,
  — OUT → GPIO22.
— **Capteur avant droit :**
  — VCC → 3.3V du Raspberry Pi,
  — GND → GND du Raspberry Pi,
  — OUT → GPIO3.
— **Capteur arrière droit :**
  — VCC → 3.3V du Raspberry Pi,
  — GND → GND du Raspberry Pi,

  - OUT → GPIO27.
- **Capteur central à l'avant :**
  - VCC → 3.3V du Raspberry Pi,
  - GND → GND du Raspberry Pi,
  - OUT → GPIO6.

Sur le schéma, chaque capteur est connecté à un GPIO différent afin de permettre au Raspberry Pi d'analyser précisément la position de la ligne et de corriger la trajectoire du robot.

Justification du nombre de capteurs

L'utilisation de cinq capteurs permet :
- une meilleure précision de détection,
- la reconnaissance des virages,
- la gestion des intersections.

Voir la partie positionnement et câblage des capteurs pour plus d'informations.

2.2.3 L293D : driver de moteurs

Rôle

Le L293D est un circuit intégré de type pont en H. Il permet :
- au Raspberry Pi, délivrant un faible courant, de commander des moteurs DC nécessitant un courant plus élevé,
- de faire fonctionner les moteurs en rotation avant, arrière ou à l'arrêt.

Connexions côté Raspberry Pi

- IN1, IN2 → GPIO23,GPIO24 (commande du sens du moteur 1),
- IN3, IN4 → GPIO27,GPIO17 (commande du sens du moteur 2),
- Enable 1 et Enable 2 → GPIO12,GPIO13 (activation et contrôle de la vitesse par PWM),
- GND → masse commune,
- Vcc1 → 5V (alimentation logique),
- Vcc2 → alimentation des moteurs (9 V)

2.2.4 Moteurs DC

Rôle

Les moteurs DC assurent le déplacement du robot, permettant les mouvements vers la gauche et la droite.

Connexions

- OUT1 / OUT2 du L293D → moteur 1,
- OUT3 / OUT4 du L293D → moteur 2.

Remarque importante

Les moteurs ne sont pas connectés directement au Raspberry Pi. Ils sont impérativement commandés via le driver L293D afin de protéger le microcontrôleur et de fournir une puissance suffisante.

2.2.5 Alimentation des moteurs (pile 9V)

Rôle

La pile 9V est utilisée pour :
— alimenter exclusivement les moteurs,
— éviter de surcharger le Raspberry Pi.

Connexions

— +9V → Vcc2 du L293D,
— − → GND commun.

Masse commune

La masse de la pile, du Raspberry Pi et du L293D est commune. Cette configuration est indispensable pour garantir une référence électrique identique et assurer le bon fonctionnement des signaux de commande.

2.2.6 Capteur de couleur TCS34725

Rôle

Le capteur TCS34725 permet :
— la détection de la couleur d'un objet ou du sol,
— la fourniture des composantes RGB.

Communication

Le capteur communique avec le Raspberry Pi via le bus I2C.

Connexions

— VCC → 3.3V,
— GND → GND,
— SDA → GPIO SDA du Raspberry Pi,
— SCL → GPIO SCL du Raspberry Pi,

Avantage

Le capteur partage le même bus I2C que l'écran LCD, ce qui permet de réduire le nombre de GPIO utilisés.

2.2.7 Écran LCD 16x2 avec interface I2C

Rôle

L'écran LCD permet l'affichage des informations du robot telles que :
— l'état du système,
— la couleur détectée,
— le mode de fonctionnement.

Connexions

— VCC → 5V,
— GND → GND,
— SDA → GPIO SDA (I2C),
— SCL → GPIO SCL (I2C).

Avantage de l'I2C

L'interface I2C nécessite uniquement deux fils de données, ce qui permet d'économiser les GPIO du Raspberry Pi.

2.2.8 Buzzer actif

Rôle

Le buzzer actif est utilisé pour émettre des signaux sonores lors :
— du démarrage du robot,
— d'alertes,
— de la détection d'événements spécifiques.

Connexions

— borne positive → GPIO18 du Raspberry Pi,
— borne négative → GND.

Remarque

Étant un buzzer actif, il ne nécessite pas de signal PWM. Un niveau logique haut (`HIGH`) appliqué au GPIO18 suffit pour produire un son.

2.3 Choix techniques et câblage

Cette section présente les principaux choix techniques réalisés lors de la conception du système, ainsi que l'organisation du câblage. Les décisions prises visent à garantir un fonctionnement fiable du robot, une protection des composants électroniques et une facilité d'intégration des différents modules.

Câblage des moteurs via le L293D

Les moteurs à courant continu ne sont pas connectés directement au Raspberry Pi. En effet, les broches GPIO du Raspberry Pi ne peuvent fournir qu'un courant limité, insuffisant pour alimenter des moteurs DC.

L'utilisation du driver de moteurs L293D, basé sur une architecture de type pont en H, permet :

— d'amplifier le courant délivré aux moteurs,
— de protéger le Raspberry Pi contre les surintensités,
— de contrôler le sens de rotation des moteurs (avant et arrière),
— de gérer la mise en marche et l'arrêt des moteurs.

Les broches d'entrée du L293D (IN1 à IN4) sont reliées aux GPIO du Raspberry Pi afin de commander le sens de rotation des moteurs, tandis que les broches *Enable* permettent l'activation et, si nécessaire, le contrôle de la vitesse par modulation de largeur d'impulsion (PWM).

Alimentation du Raspberry Pi et des autres composants

Le Raspberry Pi est alimentée indépendamment des moteurs afin d'assurer une tension stable et d'éviter les perturbations liées aux appels de courant des actionneurs. Les composants électroniques tels que les capteurs, l'écran LCD et le buzzer sont alimentés par le Raspberry Pi via les lignes 5V et 3.3V.

Les moteurs DC sont alimentés par une source externe dédiée (pile 9V), fournissant une tension mesurée d'environ 8,27 V, soit approximativement 9 V. Cette séparation des alimentations permet de limiter les chutes de tension et d'améliorer la fiabilité du système.

L'ensemble des composants partage une masse commune (GND), condition indispensable pour assurer une référence électrique cohérente et garantir le bon fonctionnement des signaux de commande.

Positionnement et câblage des capteurs

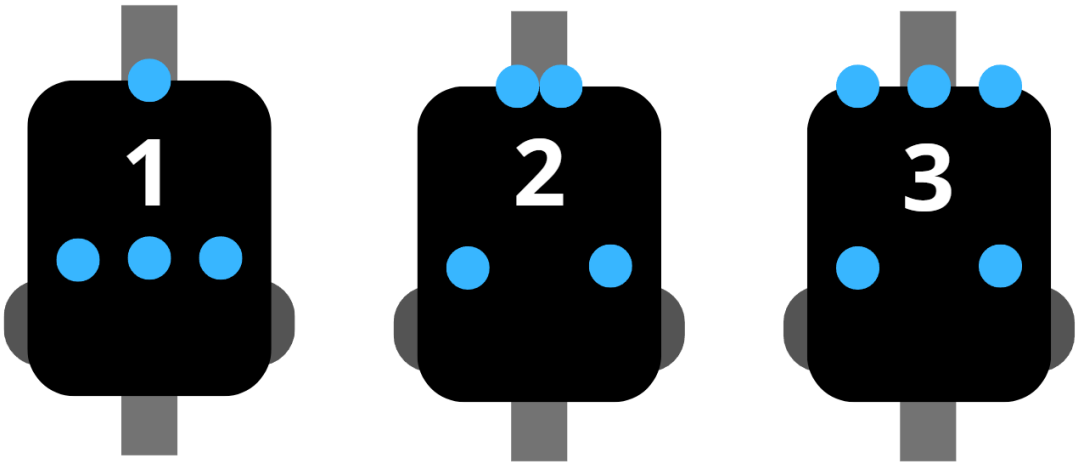


FIGURE 2.2 – Différentes phases de conception et de disposition des capteurs

Les capteurs de ligne TCRT5000 sont positionnés à l'avant du robot afin de détecter la trajectoire à suivre. Leur disposition comprend un capteur central, des capteurs latéraux

avant et des capteurs latéraux arrière, ce qui permet une détection précise de la ligne, y compris dans les virages et aux intersections. La figure 2.2 présente nos différentes phases de disposition de ces capteurs, ayant débuté par le 1 puis ayant opté pour la disposition 3, après de nombreux tests notamment au niveau du suivi de ligne. Les capteurs de détection d'intersection n'ont jamais bougé depuis notre réflexion initiale.

Chaque capteur est relié à un GPIO distinct du Raspberry Pi, permettant une lecture indépendante de chaque signal. Les fils de connexion sont organisés de manière à limiter les interférences, faciliter la maintenance et assurer une identification claire de chaque capteur.

Le capteur de couleur et l'écran LCD utilisent le bus de communication I2C, ce qui réduit le nombre de fils nécessaires et simplifie le câblage global du système.

2.4 Bilan de puissance et estimation de l'autonomie

Cette section présente l'estimation de l'autonomie du système robotique, divisée en deux circuits distincts : le circuit logique (alimenté par USB 5V) et le circuit de puissance (alimenté par les cellules Lithium 8.4V).

2.4.1 Circuit Logique (Batterie Externe USB)

Données de la source :

— Capacité nominale (C_{USB}) : 10 000 mAh

— Tension : 5 V

Consommation des composants (sous 5V) : Pour les composants fonctionnant en 3.3V (capteurs, buzzer) nous considérons le courant tiré sur le rail 5V ce qui est une approximation.

1. Raspberry Pi 4 : $I_{pi} = 1.25$ A $= 1250$ mA
2. Capteurs de ligne (x5) : $I_{ligne} = 5 \times 20$ mA $= 100$ mA
3. Capteur RGB : $I_{rgb} = 16$ mA
4. Écran LCD : $I_{lcd} = 50$ mA
5. Buzzer (activé) : $I_{buz} = 20$ mA

Le courant total consommé par la logique est calculé comme suit :

$$I_{logique} = I_{pi} + I_{ligne} + I_{rgb} + I_{lcd} + I_{buz} \tag{2.1}$$

$$I_{logique} = 1250 + 100 + 16 + 50 + 20 = 1436 \text{ mA} \tag{2.2}$$

Estimation de l'autonomie ($T_{logique}$) : Note : Les batteries externes sont souvent notées pour la capacité de leurs cellules internes (3.7V). Pour obtenir du 5V, un convertisseur est utilisé. Nous appliquons un coefficient de rendement global réaliste $\eta \approx 0.7$ (comprenant la conversion 3.7V vers 5V et les pertes thermiques).

$$T_{logique} \approx \frac{C_{USB} \times \eta}{I_{logique}} = \frac{10000 \times 0.7}{1436} \tag{2.3}$$

$$T_{logique} \approx \frac{7000}{1436} \approx \mathbf{4.87} \text{ heures} \tag{2.4}$$

2.4.2 Circuit de Puissance (Cellules Lithium)

Données de la source :

— Capacité (C_{bat}) : 3400 mAh
— Tension : 8.4 V

Consommation des composants :

1. Moteurs (x2) : $I_{mot} = 2 \times 90$ mA $= 180$ mA
2. Contrôleur L293D : $I_{driver} = 500$ mA

Le courant total du circuit de puissance est :

$$I_{puissance} = I_{mot} + I_{driver} = 180 + 500 = 680 \text{ mA} \tag{2.5}$$

Estimation de l'autonomie ($T_{puissance}$) : Pour des cellules Li-Ion en série, la capacité en mAh reste celle d'une seule cellule (3400 mAh), seule la tension s'additionne.

$$T_{puissance} = \frac{C_{bat}}{I_{puissance}} = \frac{3400}{680} \tag{2.6}$$

$$T_{puissance} \approx \mathbf{5.0} \text{ heures} \tag{2.7}$$

2.4.3 Conclusion

L'autonomie globale du système est limitée par le maillon le plus faible.

— Autonomie Logique (estimée avec pertes) : $\approx 4h52$
— Autonomie Moteurs : $\approx 5h00$

Le système devrait donc avoir une autonomie théorique d'environ **4 heures et 50 minutes** en fonctionnement continu maximal.

2.5 Difficultés rencontrées et pistes d'amélioration

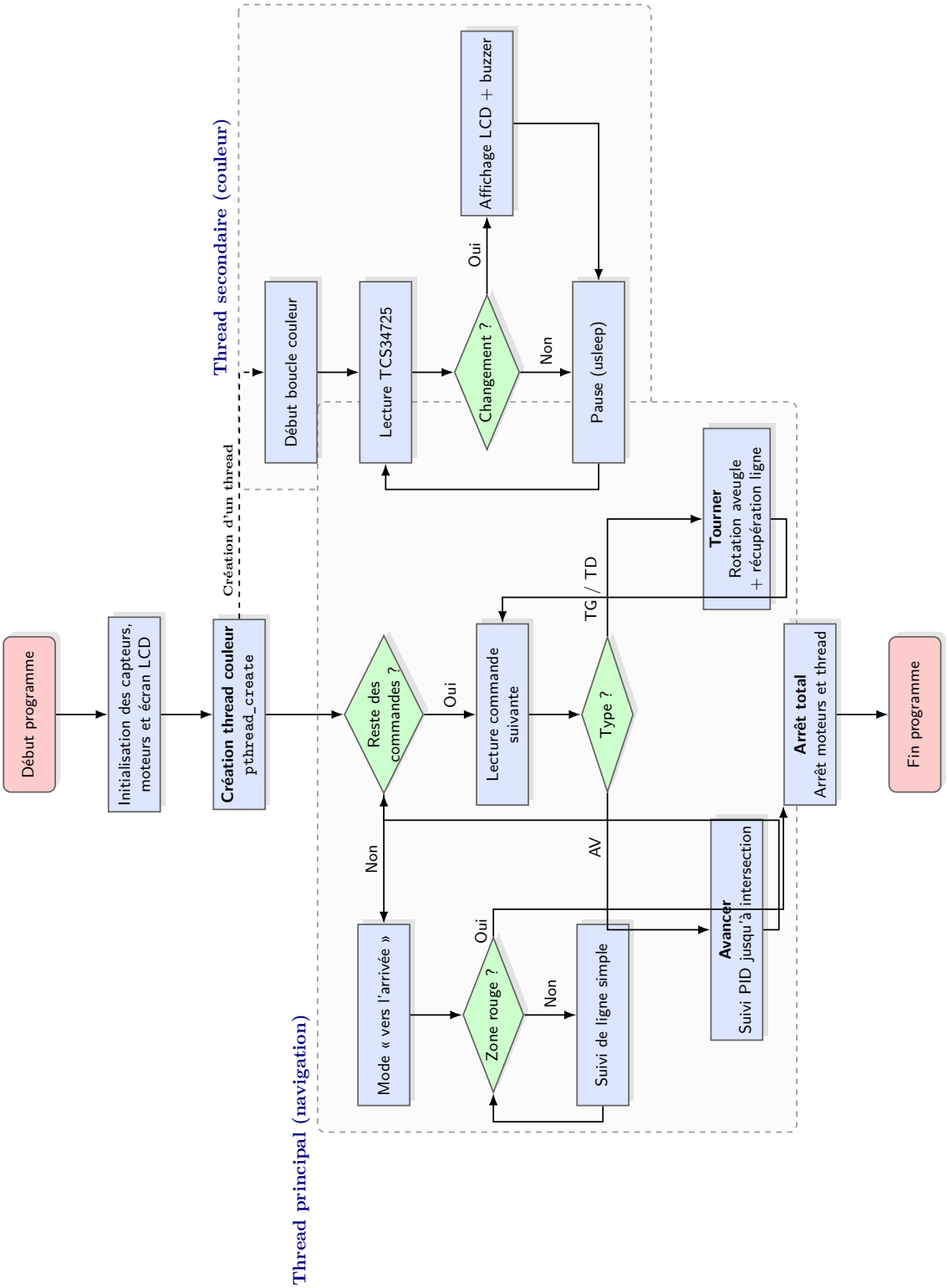
Nous avons rencontré quelques difficultés, notamment :

— Trouver un positionnement des capteurs optimal et en choisir le nombre adéquat
— Calibrer les capteurs de ligne et le capteur RGB en fonction des conditions de lumière de la salle qui ne sont jamais les mêmes.
— Brancher de manière fiable les câbles

Améliorations possibles :

— Visser les capteurs au châssis au lieu de les scotcher
— Utiliser des câbles plus longs pour mieux organiser l'intérieur du robot.

2.6 Flowchart



Chapitre 3

Partie informatique

3.0.1 Analyse

Cette phase d'analyse a pour objectif d'identifier les éléments fondamentaux nécessaires à la conception du programme. Le projet consiste à faire évoluer un robot dans un labyrinthe en lui fournissant une suite d'ordres cohérente, tout en respectant les contraintes imposées par la structure du labyrinthe et les règles de déplacement.

Avant de détailler les structures de données complexes et les algorithmes de recherche, il est nécessaire d'introduire les éléments de base utilisés dans les types abstraits de données. Ces éléments constituent le socle de la modélisation et interviennent directement dans la définition des opérations de déplacement du robot.

Types

Dans cette première étape de conception, seul le type **Direction** est introduit. Il s'agit d'une énumération regroupant les quatre directions cardinales : Nord, Est, Sud et Ouest.

Ce type permet de représenter l'orientation du robot dans le labyrinthe. L'orientation joue un rôle central dans la navigation, car elle conditionne les mouvements possibles à partir d'une position donnée. Par exemple, la notion de « case devant le robot » dépend directement de la direction dans laquelle celui-ci est orienté.

L'utilisation d'une énumération dédiée présente plusieurs avantages. Elle garantit que seules des directions valides peuvent être manipulées, améliore la lisibilité des algorithmes et facilite la définition des règles de déplacement, telles que les rotations à gauche ou à droite. Ce type est utilisé directement dans les types abstraits définis par la suite, notamment ceux liés au robot et à la gestion des états.

TAD

Avant de passer à l'analyse descendante, nous allons aborder les TAD introduits : Labyrinthe et Robot. Commençons par le TAD Labyrinthe :

Nom:	Labyrinthe	
Paramètre:		
Utilise:	Graphe⟨Sommet = NNN, Etiquette = NNN⟩, Ensemble⟨NNN⟩, Liste⟨NNN⟩, **Booleen**, **Naturel**, NNN	
Opérations:	labyrinthe:	NNN → Labyrinthe
	obtenirTaille:	Labyrinthe → NNN

obtenirNbSommets: Labyrinthe → **Naturel**
ajouterChemin: Labyrinthe × NNN × NNN → Labyrinthe
retirerChemin: Labyrinthe × NNN × NNN → Labyrinthe
estUnSommetPassage: Labyrinthe × NNN → **Booleen**
marquerSommetCommePassage: Labyrinthe × NNN → Labyrinthe
retirerSommetCommePassage: Labyrinthe × NNN → Labyrinthe
obtenirSommetsPassage: Labyrinthe → Ensemble⟨NNN⟩
obtenirSommetsAdjacents: Labyrinthe × NNN → Ensemble⟨NNN⟩
estUnChemin: Labyrinthe × NNN × NNN → **Booleen**
estUnVoyage: Labyrinthe × Liste⟨NNN⟩ → **Booleen**

Axiomes:
- $obtenirTaille(labyrinthe(t)) = t$
- $obtenirNbSommets(labyrinthe(t)) = t \times t$
- $obtenirSommetsPassage(labyrinthe(t)) = ensembleVide()$
- $estUnSommetPassage(lab, s) = estPresent(obtenirSommetsPassage(lab), s)$
- $estUnChemin(lab, s_1, s_2) = appartient(obtenirSommetsAdjacents(lab, s_1), s_2)$
- $estUnChemin(lab, s_1, s_2) = estUnChemin(lab, s_2, s_1)$
- $obtenirSommetsAdjacents(ajouterChemin(lab, s_1, s_2), s_1) = ajouter(obtenirSommetsAdjacents(lab, s_1), s_2)$
- $obtenirSommetsAdjacents(ajouterChemin(lab, s_1, s_2), s_2) = ajouter(obtenirSommetsAdjacents(lab, s_2), s_1)$
- $obtenirSommetsAdjacents(ajouterChemin(lab, s_1, s_2), s) = obtenirSommetsAdjacents(lab, s)$ si $s \neq s_1$ et $s \neq s_2$
- $obtenirSommetsAdjacents(retirerChemin(lab, s_1, s_2), s_1) = retirer(obtenirSommetsAdjacents(lab, s_1), s_2)$
- $obtenirSommetsAdjacents(retirerChemin(lab, s_1, s_2), s_2) = retirer(obtenirSommetsAdjacents(lab, s_2), s_1)$
- $estUnVoyage(lab, l) = estUnChemin(lab, obtenirElement(l, 1), obtenirElement(l, 2))$ $etestUnVoyage(lab, supprimer(l, 1))$ si $longueur(l) > 2$
- $estUnVoyage(lab, ajouter(ajouter(liste(), e_1), e_2)) = estUnChemin(lab, e_1, e_2)$

Préconditions:
retirerSommetCommePassage(lab, s): $estUnSommetPassage(lab, s)$
ajouterChemin(lab, s_1, s_2) : $non(estUnChemin(lab, s_1, s_2))$
retirerChemin(lab, s_1, s_2) : $estUnChemin(lab, s_1, s_2)$
estUnVoyage(lab, l): $longueur(l) > 1$

Ensuite on peut présenter le TAD robot, qui fonctionne conjointement avec le TAD Labyrinthe :

Nom: Robot
Paramètre:
Utilise: Labyrinthe, Direction, **Booleen**, NNN
Opérations:
robot: NNN × Direction → Robot
obtenirPosition: Robot → NNN
obtenirDirection: Robot → Direction
tournerGauche: Robot → Robot
tournerDroite: Robot → Robot
caseExistante: Robot × Labyrinthe → **Booleen**
caseSuivante: Robot × Labyrinthe → NNN

peutAvancer: Robot × Labyrinthe → **Booleen**
avancer: Robot × Labyrinthe → Robot
estSurSommetPassage: Robot × Labyrinthe → **Booleen**
peutAllerGauche: Robot × Labyrinthe → **Booleen**
peutAllerDroite: Robot × Labyrinthe → **Booleen**
directionGauche: Robot → Direction
directionDroite: Robot → Direction

Axiomes:

- $obtenirPosition(robot(c,d)) = c$
- $obtenirDirection(robot(c,d)) = d$
- $obtenirPosition(tournerGauche(r)) = obtenirPosition(r)$
- $obtenirPosition(tournerDroite(r)) = obtenirPosition(r)$
- $obtenirDirection(tournerGauche(robot(s,Nord))) = Ouest$
- $obtenirDirection(tournerGauche(robot(s,Ouest))) = Sud$
- $obtenirDirection(tournerGauche(robot(s,Sud))) = Est$
- $obtenirDirection(tournerGauche(robot(s,Est))) = Nord$
- $obtenirDirection(tournerDroite(robot(s,Nord))) = Est$
- $obtenirDirection(tournerDroite(robot(s,Est))) = Sud$
- $obtenirDirection(tournerDroite(robot(s,Sud))) = Ouest$
- $obtenirDirection(tournerDroite(robot(s,Ouest))) = Nord$
- $estSurSommetPassage(r,lab) = estPresent(obtenirSommetsPassage(lab), obtenirPosition(r))$
- $caseSuivante(robot(c,Nord),lab) = c - obtenirTaille(lab)$
- $caseSuivante(robot(c,Sud),lab) = c + obtenirTaille(lab)$
- $caseSuivante(robot(c,Est),lab) = c + 1$
- $caseSuivante(robot(c,Ouest),lab) = c - 1$
- $peutAvancer(r,lab) = caseExistante(r,lab) et\ estUnChemin(lab, obtenirPosition(r), caseSuivante(r,lab))$
- $obtenirPosition(avancer(r,lab)) = caseSuivante(r,lab)$
- $obtenirDirection(avancer(r,lab)) = obtenirDirection(r)$
- $peutAllerGauche(r,lab) = peutAvancer(tournerGauche(r),lab)$
- $peutAllerDroite(r,lab) = peutAvancer(tournerDroite(r),lab)$
- $directionGauche(r) = obtenirDirection(tournerGauche(r))$
- $directionDroite(r) = obtenirDirection(tournerDroite(r))$
- $caseExistante(r,lab) = \Big(obtenirPosition(r) \leq obtenirNbSommets(lab)\Big) ou \Big(\big(obtenirDirection(r) = Nord et obtenirPosition(r) > obtenirTaille(lab)\big) ou \big(obtenirDirection(r) = Sud et obtenirPosition(r) \leq obtenirNbSommets(lab) - obtenirTaille(lab)\big) ou \big(obtenirDirection(r) = Est and (obtenirPosition(r) \bmod obtenirTaille(lab)) \neq 0\big) ou \big(obtenirDirection(r) = Ouest and (obtenirPosition(r) \bmod obtenirTaille(lab)) \neq 1\big)\Big)$

Préconditions: :

caseSuivante(r, lab): $caseExistante(r,lab)$

avancer(r, lab): $peutAvancer(r,lab)$

Analyse descendante

Une fois ce type fondamental défini, l'analyse descendante permet de structurer la résolution du problème. L'orientation du robot, représentée par le type **Direction**, intervient dans la majorité des opérations de navigation et doit donc être intégrée dès la conception des algorithmes.

Le problème global peut être vu comme une succession de décisions dépendant à la fois de la position du robot et de son orientation. Cette observation conduit naturellement à une approche par exploration d'états, où chaque état correspond à une configuration précise du robot dans le labyrinthe.

Cette analyse constitue le fondement de la définition des types abstraits plus complexes et à la mise en place de l'algorithme de recherche utilisé pour déterminer la suite d'actions à exécuter.

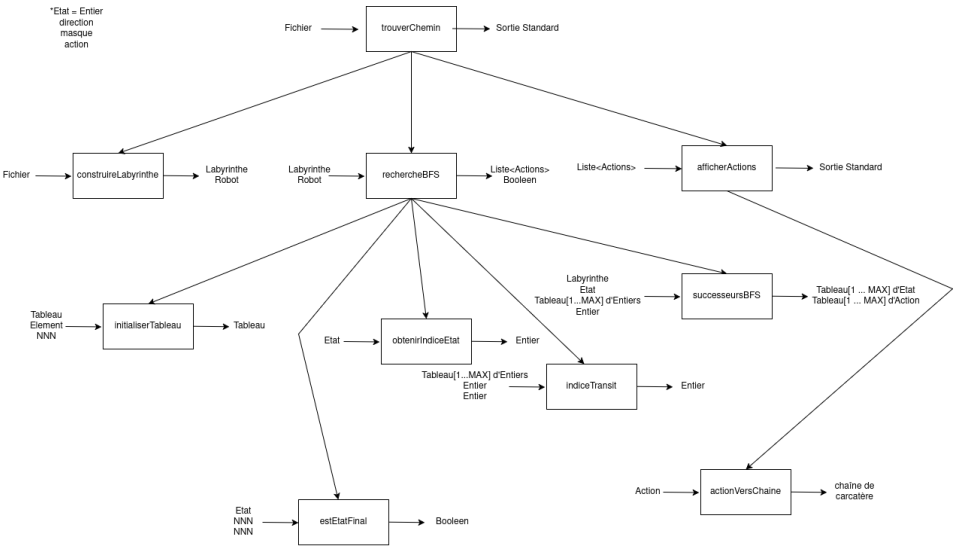


FIGURE 3.1 – Analyse Descendante

3.0.2 Conception Préliminaire

Maintenant que les TAD sont définis, nous allons voir les signatures de fonctions pour l'implémentation de nos TAD. Dans un premier temps, regardons le labyrinthe, et l'implémentation pour notre programme :

Type Labyrinthe = **Structure**
graphe : Graphe
pointsTransit : Ensemble
taille : **Entier**
finstructure

Pour ces fonctions et procédure, on va reprendre les principales opérations du TAD et utiliser les fonctions et procédures suivantes :

fonction creerLabyrinthe (taille : **Entier**) **:** Labyrinthe
fonction obtenirTaille (labyrinthe : Labyrinthe) **: Entier**
fonction obtenirPointsTransit (labyrinthe : Labyrinthe) **:** Ensemble
fonction obtenirVoisins (labyrinthe : Labyrinthe, source : **Entier**) **:** Ensemble
fonction obtenirDegre (labyrinthe : Labyrinthe, case : **Entier**) **: Entier**

procédure ajouterPassage (**E/S** labyrinthe : Labyrinthe, **E** source, destination : **Entier**)
procédure supprimerPassage (**E/S** labyrinthe : Labyrinthe, **E** source, destination : **Entier**)
procédure ajouterPointTransit (**E/S** labyrinthe : Labyrinthe, **E** source : **Entier**)
procédure supprimerPointTransit (**E/S** labyrinthe : Labyrinthe, **E** source : **Entier**)
fonction estPassage (labyrinthe : Labyrinthe, source, destination : **Entier**) **: Booleen**
fonction estTrajet (labyrinthe : Labyrinthe, trajet : Liste) **: Booleen**
fonction estPointTransit (labyrinthe : Labyrinthe, source : **Entier**) **: Booleen**
procédure afficherLabyrinthe (**E** labyrinthe : Labyrinthe)

On considère notre robot implémenté de la manière suivante :

Type Direction = {NORD, EST, SUD, OUEST}

Type Robot = **Structure**
  position : **Entier**
  direction : Direction
finstructure

Enfin on va maintenant pouvoir énoncer les différentes fonction et procédures pour le robot :

fonction creerRobot (position : **Entier**, direction : Direction) **:** Robot
fonction obtenirPosition (robot : Robot) **: Entier**
procédure definirPosition (**E/S** robot : Robot, **E** position : **Entier**)
fonction obtenirDirection (robot : Robot) **:** Direction
procédure tournerGauche (**E/S** robot : Robot)
procédure tournerDroite (**E/S** robot : Robot)
fonction caseDevantExiste (robot : Robot, laby : Labyrinthe) **: Booleen**
fonction caseSuivante (robot : Robot, laby : Labyrinthe) **: Entier**
fonction peutAvancer (robot : Robot, laby : Labyrinthe) **: Booleen**
fonction estSurPointTransit (robot : Robot, laby : Labyrinthe) **: Booleen**
procédure avancer (**E/S** robot : Robot, **E** laby : Labyrinthe)
fonction peutAllerGauche (robot : Robot, laby : Labyrinthe) **: Booleen**
fonction peutAllerDroite (robot : Robot, laby : Labyrinthe) **: Booleen**
fonction directionGauche (robot : Robot) **:** Direction
fonction directionDroite (robot : Robot) **:** Direction
fonction avancerJusqueVirage (robot : Robot, labyrinthe : Labyrinthe, positions : **Tableau[**1..N**] d'Entier**) **: Entier**

Pour ce qui est de l'algorithme liée à l'analyse descendante, on va introduire des structures nécéssaires, au bon déroulement de l'algorithme :

Type ActionRobot = {RIEN, AVANCER, TOURNER_GAUCHE, TOURNER_DROITE}

Type EtatRobot = **Structure**
  case : **Entier**
  orientation : Direction
  masque : **Entier**
  dernieereAction : ActionRobot
finstructure

Type Successeur = **Structure**
  action : ActionRobot
  etat : EtatRobot
finstructure

Type ListeSuccesseurs = **Structure**
  elements : **Tableau[**1..N**] de**Successeur
  nombre : **Entier**
finstructure

Type InfoVisite = **Structure**
  distance : **Entier**
  indiceParent : **Entier**
  actionParent : ActionRobot
finstructure

Type ResultatBFS = **Structure**
  actions : Liste
  trouve : **Booleen**
finstructure

fonction construireLabyrinthe (nomFichier : **Chaine de caracteres**, **E/S** labyrinthe : Labyrinthe, **E/S** robot : Robot) **: Booleen**
fonction successeursBFS (labyrinthe : Labyrinthe, etat : EtatRobot, pointsTransit : **Tableau[**1..MAX**] d'Entier**, nbPoints : **Entier) :** ListeSuccesseurs
fonction rechercheBFS (labyrinthe : Labyrinthe, robot : Robot) **:** ResultatBFS
procédure afficherActions (**E** actions : Liste, **E** fichierSortie : Fichier)

Pour faire un point sur les différentes structures, l'important est de retenir que l'on peut stocker l'état d'un robot à un instant T (position, direction, masque, dernière action). Ainsi on peut également avec Successeur noter le nouvel état d'un robot associé à une action. Enfin on peut stocker une liste de ces successeurs. Pour le parcours, infoVisite est une structure qui va noter la distance et l'indice parent associé à un état, cette structure sera très utile pour le backtracking. Enfin une structure de résultat est présente par confort pour l'implémentation principalement.

Afin d'expliquer le fonctionnement de l'algorithme, que l'on va implémenter, on va réaliser un parcours en profondeur des états du Robot. Concrètement, pour faire le BFS :
— On initialise une file avec l'état initial (défini par le robot)
— Tant que l'on à pas fait le parcours, on défile un état de la file
— On calcule tous les nouveaux états possible non visités à partir de cet état
— Une fois un état final trouvé, on remonte les états du point d'arrivée vers le point de départ pour reconstruire le schéma

3.1 Conception détaillée

3.1.1 Fonctions du Labyrinthe

fonction creerLabyrinthe (n : **Entier**) **:** Labyrinthe

Déclaration n : NNN
L : Labyrinthe
i : **Entier**

debut
L.taille ← n
L.g ← graphe()
L.passages ← ensemble()
pour i ←1 **à** n*n **faire**
ajouterSommet(L.g, i)
finpour
retourner L
fin

fonction estTrajet (L : Labyrinthe, trajet : Liste<NNN>) **: Booleen**

Déclaration u,v : NNN
i : **Entier**

debut
si Liste.longueur(trajet) < 2 **alors**
retourner faux
finsi
pour i ←1 **à** longueur(trajet)-1 **faire**
u ← obtenirElement(trajet, i)
v ← obtenirElement(trajet, i+1)
si non estChemin(L,u,v) **alors**
retourner faux
finsi
finpour
retourner vrai
fin

3.1.2 Fonctions du Robot

fonction creerRobot (pos : **Entier**, dir : Direction) **:** Robot

Déclaration R : Robot

debut
R.position ← pos
R.direction ← dir
retourner R
fin

fonction caseDevantExiste (R : Robot, L : Labyrinthe) **: Booleen**

```
   Déclaration  n : Entier
                p : Entier
                lig, col : Entier
debut
   n ← obtenirTaille(L)
   p ← obtenirPosition(R)
   lig ← ((p-1) div n)+1
   col ← ((p-1) mod n)+1
   si R.direction = NORD alors
      retourner lig > 1
   finsi
   si R.direction = SUD alors
      retourner lig < n
   finsi
   si R.direction = EST alors
      retourner col < n
   finsi
   // OUEST
   retourner col > 1
fin

fonction caseSuivante (R : Robot, L : Labyrinthe) : Entier
   Déclaration  n : Entier
                p : Entier
debut
   n ← L.taille
   p ← R.position
   si R.direction = NORD alors
      retourner p - n
   finsi
   si R.direction = SUD alors
      retourner p + n
   finsi
   si R.direction = EST alors
      retourner p + 1
   finsi
   // OUEST
   retourner p - 1
fin

fonction peutAvancer (R : Robot, L : Labyrinthe) : Booleen
   Déclaration  p : Entier
                q : Entier
debut
   si non caseDevantExiste(R,L) alors
      retourner faux
   finsi
```

```
   p ← R.position
   q ← caseSuivante(R,L)
   retourner estUnChemin(L, p, q)
fin

procédure avancer ( E/S R : Robot E L : Labyrinthe )
debut
   si peutAvancer(R,L) alors
      R.position ← caseSuivante(R,L)
   finsi
fin

fonction avancerJusqueVirage (R : Robot, L : Labyrinthe) : Tableau[1..MAX] de NNN,
Entier
   Déclaration   chemin : Tableau[1..MAX] de NNN
                 i : Entier
debut
   i ← 1
   tant que peutAvancer(R,L) faire
      avancer(R,L)
      chemin[i] ← obtenirPosition(R)
      // Stop si on arrive à un embranchement (degré ≠ 2)
      si obtenirDegre(L, R.position) ≠ 2 alors
         retourner chemin, i
      finsi
      i ← i+1
   fintantque
   retourner chemin, i
fin
```

3.1.3 Algorithme générale

```
fonction InfoVisiteInvalide () : InfoVisite
   Déclaration   iv : InfoVisite
debut
   iv.distance ← -1
   iv.indiceParent ← -1
   iv.actionParent ← RIEN
   retourner iv
fin

procédure initialiserTableau ( E/S t : Tableau[0..MAX] deT E val : T, n : Entier )
   Déclaration   i : Entier
debut
   pour i ←1 à n faire
```

```
        t[i] ← val
    finpour
fin

fonction indexEtat (e : EtatRobot, nbMasques : NNN) : Entier
    Déclaration  o : Entier
                 idx : Entier
                 c0 : Entier
debut
    // Encodage de l'orientation en 0..3
    si e.orientation = NORD alors
        o ← 0
    finsi
    si e.orientation = EST alors
        o ← 1
    finsi
    si e.orientation = SUD alors
        o ← 2
    finsi
    si e.orientation = OUEST alors
        o ← 3
    finsi
    c0 ← e.case - 1
    // Indexation : ((case * 4) + orientation) * nbMasques + masque
    idx ← c0
    idx ← idx * 4 + o
    idx ← idx * nbMasques + e.masque
    retourner idx
fin

fonction estEtatFinal (e : EtatRobot, masqueBut, nbMasques : NNN) : Booleen
debut
    si e.masque < 0 alors
        retourner faux
    finsi
    si e.masque ≥ nbMasques alors
        retourner faux
    finsi
    retourner e.masque = masqueBut
fin

fonction successeursBFS ( lab : Labyrinthe, e : EtatRobot, points : Tableau[1..nbPoints]
de NNN, nbPoints : Entier) : ListeSuccesseurs
    Déclaration  succ : ListeSuccesseurs
                 r, r2 : Robot
                 e2 : EtatRobot
                 chemin : Tableau[1..MAX] de NNN
```

```
                k : Entier
                i, j : Entier
debut
   // Reconstruire le robot depuis l'état
   r ← robot(e.case, e.orientation)
   // 1) Successeur : AVANCER (condensé jusqu'au virage)
   si peutAvancer(r, lab) alors
      // On avance le robot r en place; chemin contient les cases traversées
      r2 ← r
      (chemin, k) ← avancerJusqueVirage(r2, lab)
      // Construire le nouvel état e2
      e2.case ← obtenirPosition(r)+k
      e2.orientation ← obtenirDirection(r)
      e2.masque ← e.masque
      e2.derniereAction ← AVANCER
      // Mise à jour du masque : on marque tous les points atteints sur le chemin
      pour j ←1 à k faire
         pour i ←1 à nbPoints faire
            si chemin[j] = points[i] alors
            e2.masque ← e2.masque OR (1 ≪ (i − 1))
            finsi
         finpour
      finpour
      ListeSuccesseurs.ajouter(succ, AVANCER, e2)
   finsi
   // 2) Successeur : TOURNER_ GAUCHE
   r2 ← tournerGauche(r)
   e2.case ← e.case
   e2.orientation ← obtenirDirection(r2)
   e2.masque ← e.masque
   e2.derniereAction ← TOURNER_GAUCHE
   ListeSuccesseurs.ajouter(succ, TOURNER_GAUCHE, e2)
   // 3) Successeur : TOURNER_ DROITE
   r2 ← tournerDroite(r)
   e2.case ← e.case
   e2.orientation ← obtenirDirection(r2)
   e2.masque ← e.masque
   e2.derniereAction ← TOURNER_DROITE
   ListeSuccesseurs.ajouter(succ, TOURNER_DROITE, e2)
   retourner succ
fin
```

Il est important de détailler le fonctionnement du masque qui peut être légèrement contre-intuitif, le rôle du masque est de faire un suivi des cases visitées. Concrètement on va utiliser la notation binaire d'un nombre pour suivre cela. Admettons que nous ayons 3 points de passages à traverser, on va donc prendre un nombre sur 3 bits. Et passer le premier bit à 1 lorsque nous avons traversé le premier sommet, le deuxième pour le deuxième sommet, etc ... Cette solution a le mérite d'être très légère que ce soit en calcul

ou en mémoire pour notre exploration.

fonction rechercheBFS (L : Labyrinthe, R : Robot) **:** ResultatBFS

Déclaration file : File<EtatRobot>
visites : **Tableau[**1..MAX**] de**InfoVisite
vu : **Tableau[**1..MAX**] deBooleen**
etat0, etat, s : EtatRobot
idx0, idx, idxs, taille : **Entier**
trouve : **Booleen**
idxFinal : **Entier**
succ : ListeSuccesseurs
pile : Pile<ActionRobot>
act : ActionRobot
actions : Liste<ActionRobot>
res : ResultatBFS

debut

// Initialisations
file ← File.file()
actions ← Liste.liste()
pile ← Pile.pile()
trouve ← faux
idxFinal ← -1
taille ← 16*obtenirTaille(L)*obtenirTaille(L)*(puissance(2, cardinalite(obtenirPointsTransit)-1)
initialiserTableau(vu, faux, taille)
initialiserTableau(visites, InfoVisiteInvalide(), taille)

// État initial
etat0.case ← obtenirPosition(R)
etat0.orientation ← obtenirDirection(R)
etat0.masque ← 0
etat0.derniereAction ← RIEN
idx0 ← indexEtat(etat0)
File.enfiler(file, etat0)
vu[idx0] ← vrai
visites[idx0].distance ← 0
visites[idx0].indiceParent ← -1
visites[idx0].actionParent ← RIEN

// BFS
tant que non File.estVide(file) **faire**
  etat ← File.defiler(file)
  idx ← indexEtat(etat)
  si estEtatFinal(etat, L) **alors**
    trouve ← vrai
    idxFinal ← idx
    File.vider(file)
  finsi

```
        // Successeurs
        succ ← successeursBFS(L, etat)
        pour i ←1 à succ.nombre faire
            s ← succ.elements[i].etat
            idxs ← indexEtat(s)
            si non vu[idxs] alors
                vu[idxs] ← vrai
                visites[idxs].distance ← visites[idx].distance + 1
                visites[idxs].indiceParent ← idx
                visites[idxs].actionParent ← succ.elements[i].action
                File.enfiler(file, s)
            finsi
        finpour
    fintantque

    // Reconstruction avec une pile
    si trouve alors
        idx ← idxFinal
        tant que visites[idx].indiceParent ≠ -1 faire
            act ← visites[idx].actionParent
            Pile.empiler(pile, act)
            idx ← visites[idx].indiceParent
        fintantque
        tant que non Pile.estVide(pile) faire
            act ← Pile.depiler(pile)
            Liste.inserer(actions, act, Liste.longueur(actions)
        fintantque
    finsi
    res.actions ← actions
    res.trouve ← trouve
    retourner res
fin
```

3.2 Implémentation en C

3.2.1 Organisation des fichiers

Le répertoire principal `programmes/` regroupe l'ensemble des fichiers liés au développement logiciel du projet. Il est organisé en deux sous-répertoires principaux correspondant aux deux grandes parties du système : la partie algorithmique et la partie électronique.

```
programmes/
 algo/
 elec/
```

Cette organisation permet une séparation claire entre la logique de traitement et la gestion du matériel.

Répertoire `algo/` — Partie algorithmique Le répertoire `algo/` contient l'ensemble des fichiers relatifs à la logique algorithmique du projet, indépendamment des contraintes matérielles.

```
algo/
 bin/
 include/
 include_lib/
 lib/
 src/
 src_lib/
 Doxyfile
 makefile
 ville.txt
```

— `bin/` : contient les fichiers exécutables générés après compilation de la partie algorithmique.
— `src/` : regroupe les fichiers sources en langage C (`.c`) implémentant les algorithmes principaux du projet (logique de décision, calculs, traitements).
— `include/` : contient les fichiers d'en-tête (`.h`) associés aux sources de `src/`. Ces fichiers définissent les prototypes des fonctions, les structures de données et les constantes utilisées.
— `src_lib/` : regroupe les fichiers sources des bibliothèques internes développées pour la partie algorithmique. Ces bibliothèques contiennent des fonctions génériques et réutilisables.
— `include_lib/` : contient les fichiers d'en-tête associés aux bibliothèques présentes dans `src_lib/`. Il permet une séparation claire entre le code applicatif et le code de bibliothèque.
— `lib/` : regroupe les bibliothèques compilées (par exemple fichiers `.a` ou `.so`) issues des sources de `src_lib/`.
— `Doxyfile` : fichier de configuration utilisé par l'outil *Doxygen* pour générer automatiquement la documentation du code source.
— `makefile` : permet d'automatiser la compilation de la partie algorithmique, en gérant les dépendances, la génération des bibliothèques et des exécutables.
— `ville.txt` : fichier de données utilisé comme entrée par les algorithmes (paramètres, description de l'environnement ou scénarios de test).

Répertoire `elec/` — Partie électronique Le répertoire `elec/` regroupe les fichiers liés à la gestion du matériel électronique et à l'interfaçage avec les composants physiques du système.

```
elec/
 bin/
 include/
 src/
 tests/
 deploy.sh
 makefile
```

— `bin/` : contient les exécutables générés après compilation de la partie électronique.
— `src/` : regroupe les fichiers sources en langage C dédiés à la gestion du matériel (capteurs, actionneurs, communication avec les composants électroniques).
— `include/` : contient les fichiers d'en-tête définissant les interfaces des modules électroniques, les constantes matérielles et les prototypes de fonctions.
— `tests/` : regroupe les programmes de test permettant de valider le bon fonctionnement des différents modules électroniques de manière indépendante.
— `deploy.sh` : script permettant d'automatiser le déploiement du programme sur la cible matérielle (copie des fichiers, installation et exécution).
— `makefile` : gère la compilation spécifique à la partie électronique, en tenant compte des dépendances matérielles et des bibliothèques nécessaires.

Fichiers à la racine du répertoire `programmes/` En plus des sous-répertoires `algo/` et `elec/`, le répertoire `programmes/` contient deux fichiers assurant la coordination globale du projet.
— `makefile` : ce fichier permet d'orchestrer la compilation complète du projet. Il appelle les `makefile` présents dans les sous-répertoires `algo/` et `elec/`, assurant ainsi une compilation cohérente de l'ensemble des modules du système. Il facilite également la gestion des dépendances entre les différentes parties du projet.
— `run.sh` : script d'exécution permettant de lancer automatiquement le programme après compilation. Il peut inclure des étapes telles que la vérification de la présence des exécutables, le choix du mode d'exécution ou le lancement séquentiel des différentes parties du projet.

Bilan Cette organisation en deux sous-projets distincts permet une séparation claire entre la logique algorithmique et la gestion du matériel. Elle améliore la lisibilité du code, facilite la maintenance et favorise une approche modulaire et évolutive du développement.

3.2.2 Tests unitaires

Tests unitaires

Les fonctions essentielles du projet ont été validées à l'aide de tests unitaires implémentés en langage C, en s'appuyant sur la bibliothèque `CUnit`. Ces tests ont permis de vérifier le bon fonctionnement des modules de manière isolée, avant leur intégration dans le système global.

Deux ensembles de tests ont été réalisés :
— des tests unitaires du module `labyrinthe`,
— des tests unitaires du module `robot`.

Chaque ensemble de tests est organisé sous forme d'une suite de tests CUnit, avec une initialisation et un nettoyage dédiés.

—

Outil de test utilisé La bibliothèque `CUnit` a été utilisée pour la mise en place des tests unitaires. Elle permet :
— la création de suites de tests,
— l'exécution automatique des tests,
— la vérification des résultats à l'aide d'assertions,

— l'affichage détaillé des erreurs en cas d'échec.
Les tests sont exécutés en mode verbeux afin de faciliter l'analyse des résultats.

Tests unitaires du module `labyrinthe` Les tests du module `labyrinthe` visent à vérifier la cohérence de la structure de données représentant le labyrinthe ainsi que les opérations associées.

— **Création du labyrinthe** : le test vérifie que la fonction `creerLabyrinthe` alloue correctement la structure, initialise la taille du labyrinthe et que les ensembles internes (points de transit) sont vides après la création.
— **Ajout et vérification des passages** : le test s'assure que l'ajout d'un passage entre deux cases crée bien un lien non orienté, vérifiable dans les deux sens à l'aide de la fonction `estPassage`.
— **Suppression des passages** : ce test valide que la suppression d'un passage retire correctement la connexion entre deux cases, dans les deux directions.
— **Gestion des points de transit** : le test vérifie l'ajout et la suppression de points de transit, ainsi que la mise à jour correcte de leur cardinalité et de leur présence dans l'ensemble associé.
— **Obtention des voisins** : ce test contrôle que la fonction `obtenirVoisins` retourne exactement les cases adjacentes accessibles à partir d'une case donnée.
— **Validation d'un trajet** : la fonction `estTrajet` est testée sur :
  — un trajet valide (suite de cases connectées),
  — un trajet invalide (passage inexistant),
  — un trajet vide.
  Le test vérifie que la continuité du chemin est correctement détectée.

Tests unitaires du module `robot` Les tests du module `robot` permettent de vérifier le comportement du robot dans le labyrinthe, en termes de position, d'orientation et de déplacement.

— **Création du robot et accesseurs** : ce test vérifie l'initialisation correcte du robot, ainsi que le bon fonctionnement des fonctions permettant d'accéder et de modifier sa position et sa direction.
— **Rotations et directions relatives** : les fonctions de rotation à droite et à gauche sont testées afin de vérifier la cohérence des orientations successives. Les fonctions `directionGauche` et `directionDroite` sont également validées.
— **Gestion des limites du labyrinthe** : ce test vérifie que le robot ne peut pas accéder à une case inexistante lorsqu'il se trouve au bord du labyrinthe.
— **Déplacement et gestion des passages** : le test s'assure que le robot ne peut pas avancer en l'absence de passage, et qu'il se déplace correctement lorsque le passage existe.
— **Détection des points de transit** : ce test valide la détection correcte des points de transit lorsque le robot se trouve sur une case marquée comme telle.
— **Anticipation des mouvements latéraux** : les fonctions permettant de vérifier la possibilité de déplacement à gauche ou à droite sont testées sans modifier l'état du robot.
— **Avancée automatique jusqu'à un virage** : ce test vérifie que la fonction `avancerJusqueVirage` permet au robot d'avancer automatiquement le long d'un couloir jusqu'à atteindre un point d'arrêt ou de changement de direction, tout en mémorisant le chemin parcouru.

Validation et résultats L'ensemble des tests unitaires a été exécuté avec succès. Les assertions utilisées permettent de détecter immédiatement toute incohérence dans le comportement des fonctions testées.

Ces tests ont permis de valider le bon fonctionnement des modules critiques du projet avant leur intégration dans le programme global.

Annexe – Pseudo-code des tests unitaires (CUnit)

Fichier 1 : Tests du module `labyrinthe`

```
INCLURE stdio.h
INCLURE CUnit/Basic.h
INCLURE "labyrinthe.h"
INCLURE "copieLiberationComparaisonTypesSimples.h"

FONCTION init_suite_success() : retourne 0
FONCTION clean_suite_success() : retourne 0

TEST test_creer_labyrinthe():
    laby <- creerLabyrinthe(4)
    ASSERT_PTR_NOT_NULL_FATAL(laby)
    ASSERT_EQUAL(obtenirTaille(laby), 4)
    ASSERT_TRUE(THE_estVide(*(obtenirPointsTransit(laby))))
    libererLabyrinthe(laby)

TEST test_ajouter_et_verifier_passage():
    laby <- creerLabyrinthe(2)
    ajouterPassage(laby, 1, 2)
    ASSERT_TRUE(estPassage(laby, 1, 2))
    ASSERT_TRUE(estPassage(laby, 2, 1))
    libererLabyrinthe(laby)

TEST test_supprimer_passage():
    laby <- creerLabyrinthe(2)
    ajouterPassage(laby, 1, 2)
    supprimerPassage(laby, 1, 2)
    ASSERT_FALSE(estPassage(laby, 1, 2))
    ASSERT_FALSE(estPassage(laby, 2, 1))
    libererLabyrinthe(laby)

TEST test_transit_points():
    laby <- creerLabyrinthe(3)
    ajouterPointTransit(laby, 3)
    ajouterPointTransit(laby, 5)
    ASSERT_TRUE(estPointTransit(laby, 3))
    ASSERT_TRUE(estPointTransit(laby, 5))
    ASSERT_EQUAL(THE_cardinalite(*(obtenirPointsTransit(laby))), 2)
    supprimerPointTransit(laby, 3)
    ASSERT_FALSE(estPointTransit(laby, 3))
```

```
    ASSERT_EQUAL(THE_cardinalite(*(obtenirPointsTransit(laby))), 1)
    libererLabyrinthe(laby)

TEST test_obtenir_voisins():
    laby <- creerLabyrinthe(3)
    ajouterPassage(laby, 1, 2)
    ajouterPassage(laby, 1, 4)
    voisins <- obtenirVoisins(laby, 1)
    voisinA <- 2
    voisinB <- 4
    nonVoisin <- 5
    ASSERT_EQUAL(THE_cardinalite(voisins), 2)
    ASSERT_TRUE(THE_estPresent(voisins, &voisinA))
    ASSERT_TRUE(THE_estPresent(voisins, &voisinB))
    ASSERT_FALSE(THE_estPresent(voisins, &nonVoisin))
    THE_vider(&voisins)
    libererLabyrinthe(laby)

TEST test_est_trajet():
    laby <- creerLabyrinthe(3)
    ajouterPassage(laby, 1, 2)
    ajouterPassage(laby, 2, 3)

    valide <- LCL_liste(CLCTS_copierInt, CLCTS_libererInt)
    n1 <- 1 ; n2 <- 2 ; n3 <- 3
    LCL_inserer(&valide, &n1, 0)
    LCL_inserer(&valide, &n2, 1)
    LCL_inserer(&valide, &n3, 2)
    ASSERT_TRUE(estTrajet(laby, &valide))

    invalide <- LCL_liste(CLCTS_copierInt, CLCTS_libererInt)
    n4 <- 4
    LCL_inserer(&invalide, &n1, 0)
    LCL_inserer(&invalide, &n4, 1)
    ASSERT_FALSE(estTrajet(laby, &invalide))

    vide <- LCL_liste(CLCTS_copierInt, CLCTS_libererInt)
    ASSERT_FALSE(estTrajet(laby, &vide))

    LCL_vider(&valide)
    LCL_vider(&invalide)
    LCL_vider(&vide)
    libererLabyrinthe(laby)

PROGRAMME PRINCIPAL main(argc, argv):
    initialiser registre de tests CUnit
    ajouter suite "Tests boite noire : labyrinthe"
    ajouter tests:
```

```
        - test_creer_labyrinthe
        - test_ajouter_et_verifier_passage
        - test_supprimer_passage
        - test_transit_points
        - test_obtenir_voisins
        - test_est_trajet
    lancer les tests en mode verbeux
    afficher les échecs
    nettoyer registre CUnit
    retourner code d'erreur
```

Fichier 2 : Tests du module `robot`

```
INCLURE stdio.h
INCLURE stdlib.h
INCLURE CUnit/Basic.h
INCLURE "robot.h"
INCLURE "labyrinthe.h"

FONCTION init_suite_robot() : retourne 0
FONCTION clean_suite_robot() : retourne 0

TEST test_creation_et_getters():
    robot <- creerRobot(5, NORD)
    ASSERT_PTR_NOT_NULL_FATAL(robot)
    ASSERT_EQUAL(obtenirPosition(robot), 5)
    ASSERT_EQUAL(obtenirDirection(robot), NORD)
    definirPosition(robot, 10)
    ASSERT_EQUAL(obtenirPosition(robot), 10)
    libererRobot(robot)

TEST test_rotations():
    robot <- creerRobot(1, NORD)

    tournerDroite(robot)  -> ASSERT direction == EST
    tournerDroite(robot)  -> ASSERT direction == SUD
    tournerDroite(robot)  -> ASSERT direction == OUEST
    tournerDroite(robot)  -> ASSERT direction == NORD

    tournerGauche(robot)  -> ASSERT direction == OUEST

    ASSERT_EQUAL(directionGauche(robot), SUD)
    ASSERT_EQUAL(directionDroite(robot), NORD)

    libererRobot(robot)

TEST test_limites_grille():
    laby <- creerLabyrinthe(3)
    robot <- creerRobot(2, NORD)
```

```
    ASSERT_FALSE(caseDevantExiste(robot, laby))

    definirPosition(robot, 4)
    tournerGauche(robot)  // direction OUEST
    ASSERT_FALSE(caseDevantExiste(robot, laby))

    definirPosition(robot, 5)
    ASSERT_TRUE(caseDevantExiste(robot, laby))
    ASSERT_EQUAL(caseSuivante(robot, laby), 4)

    libererLabyrinthe(laby)
    libererRobot(robot)

TEST test_mouvement_et_passages():
    laby <- creerLabyrinthe(3)
    robot <- creerRobot(1, EST)

    ASSERT_FALSE(peutAvancer(robot, laby))
    ASSERT_FALSE(avancer(robot, laby))
    ASSERT_EQUAL(obtenirPosition(robot), 1)

    ajouterPassage(laby, 1, 2)
    ASSERT_TRUE(peutAvancer(robot, laby))
    ASSERT_TRUE(avancer(robot, laby))
    ASSERT_EQUAL(obtenirPosition(robot), 2)

    libererLabyrinthe(laby)
    libererRobot(robot)

TEST test_points_transit_robot():
    laby <- creerLabyrinthe(3)
    robot <- creerRobot(5, NORD)

    ASSERT_FALSE(estSurPointTransit(robot, laby))
    ajouterPointTransit(laby, 5)
    ASSERT_TRUE(estSurPointTransit(robot, laby))

    libererLabyrinthe(laby)
    libererRobot(robot)

TEST test_regarder_cote():
    laby <- creerLabyrinthe(3)
    robot <- creerRobot(5, NORD)

    ajouterPassage(laby, 5, 4)
    ASSERT_TRUE(peutAllerGauche(robot, laby))
    ASSERT_FALSE(peutAllerDroite(robot, laby))
```

```
    ASSERT_EQUAL(obtenirPosition(robot), 5)
    ASSERT_EQUAL(obtenirDirection(robot), NORD)

    libererLabyrinthe(laby)
    libererRobot(robot)

TEST test_avancer_jusque_virage():
    laby <- creerLabyrinthe(5)
    robot <- creerRobot(1, EST)
    chemin <- NULL

    ajouterPassage(laby, 1, 2)
    ajouterPassage(laby, 2, 3)
    ajouterPassage(laby, 3, 4)

    nb_pas <- avancerJusqueVirage(robot, laby, &chemin)

    ASSERT_EQUAL(nb_pas, 3)
    ASSERT_PTR_NOT_NULL(chemin)
    SI chemin != NULL:
        ASSERT_EQUAL(chemin[0], 2)
        ASSERT_EQUAL(chemin[1], 3)
        ASSERT_EQUAL(chemin[2], 4)
        free(chemin)

    ASSERT_EQUAL(obtenirPosition(robot), 4)

    libererLabyrinthe(laby)
    libererRobot(robot)

PROGRAMME PRINCIPAL main():
    initialiser registre de tests CUnit
    ajouter suite "Tests Robot"
    ajouter tests:
        - test_creation_et_getters
        - test_rotations
        - test_limites_grille
        - test_mouvement_et_passages
        - test_points_transit_robot
        - test_regarder_cote
        - test_avancer_jusque_virage
    lancer les tests en mode verbeux
    afficher les échecs
    nettoyer registre CUnit
    retourner code d'erreur
```

3.2.3 Documentation

Afin de documenter le code source du projet *smartrobot*, une documentation automatique a été générée à l'aide de l'outil `Doxygen`. Cette approche permet de produire une documentation claire et cohérente, directement à partir des fichiers d'en-tête du projet.

Objectif de la documentation La documentation générée vise à décrire l'API du projet, en particulier les modules `labyrinthe` et `robot`. Elle facilite la compréhension des structures de données et des fonctions mises à disposition, aussi bien pour l'utilisation que pour la maintenance du code.

Configuration de Doxygen La génération de la documentation est pilotée par un fichier de configuration `Doxyfile`, dont le contenu a été volontairement simplifié afin d'éviter toute interaction lors de l'exécution.

Le projet est identifié par les paramètres suivants :
— `PROJECT_NAME` : définit le nom du projet (*smartrobot*),
— `PROJECT_BRIEF` : fournit une description synthétique de l'API (*labyrinthe et robot*).

La documentation est optimisée pour le langage C, grâce à l'option `OPTIMIZE_OUTPUT_FOR_C`, et l'ensemble des entités est extrait, y compris celles ne disposant pas de commentaires explicites (`EXTRACT_ALL`).

Périmètre analysé L'analyse du code est volontairement limitée aux fichiers d'en-tête du projet :
— le répertoire `include/` est spécifié comme source d'entrée,
— seuls les fichiers `.h` sont pris en compte,
— l'analyse est récursive afin d'inclure les éventuels sous-répertoires.

Ce choix permet de documenter l'interface publique du projet, sans exposer les détails d'implémentation internes.

Formats de sortie La documentation est générée sous deux formats complémentaires :
— un format **HTML**, permettant une consultation interactive via un navigateur web ;
— un format **LaTeX**, pouvant être compilé afin d'obtenir un document PDF.

Les fichiers générés sont placés dans le répertoire `doc/`.

Génération de la documentation La génération de la documentation est réalisée à l'aide de la commande suivante, exécutée depuis le répertoire contenant le fichier `Doxyfile` :

```
doxygen Doxyfile
```

L'exécution est silencieuse (`QUIET = YES`) et les avertissements liés à l'absence de commentaires ne sont pas bloquants, ce qui garantit une génération fluide de la documentation.

Bilan L'utilisation de Doxygen permet de maintenir une documentation automatiquement synchronisée avec le code source. Cette démarche améliore la lisibilité du projet, facilite sa prise en main et contribue à une meilleure qualité globale du développement.

Chapitre 4

Démonstration

4.1 Procédure de lancement

1. Se connecter en SSH sur la Raspberry Pi ;
2. Cloner le dépôt Git ;
3. Build le projet (via `make`) ;
4. Positionner le robot sur la pastille rouge dans le bon sens ;
5. Exécuter le script `run.sh` qui exécutera les deux programmes (cf. explications précédentes).

4.2 Mesure du temps

```
algo git:(Algo-elec) time ./bin/laby ville.txt
AV
TG
AV
[...]
TD
AV
AV
.
./bin/laby ville.txt  0,06s user 0,00s system 98% cpu 0,060 total
```

L'exécution de l'algorithme est très rapide : 60 millisecondes pour la ville du sujet.

Pour la mesure du temps du robot, nous n'avons pas réussi à terminer le parcours le jour de la soutenance. Par conséquent, nous n'avons pas de métrique à ce sujet. Néanmoins, nous avons réussi à parcourir les 15 premières cases en 30 secondes.

4.3 Comportement observé

4.3.1 Première tentative

Lors de la première, le robot a détecté la ligne entre deux plaques comme une intersection, il s'est alors mis à tourner au milieu d'une ligne droite.

4.3.2 Deuxième tentative

Lors de la deuxième tentative, le robot a parcouru sans faute les 15 premières tuiles. Il a sonné correctement au passage des pastilles, il a bien affiché toutes les instructions sur son écran. Néanmoins, à la sortie d'un virage qui était suivi d'une pastille, les capteurs n'ont pas lu la ligne noire qui était cachée par la pastille.

4.3.3 Troisième tentative

Lors de la troisième tentative, le robot s'est planté dès le premier virage pour la même raison, les capteurs avant ont besoin de passer sur la ligne noire pour arrêter la rotation mais la pastille verte cachait la ligne...

Conclusion

Ce projet intégratif nous a permis de mettre en œuvre de manière concrète les compétences acquises en électronique pour les systèmes embarqués et en algorithmique avancée et programmation C. L'objectif était ambitieux : concevoir un système autonome capable d'analyser une ville virtuelle, de calculer le chemin optimal et de s'y déplacer physiquement avec précision.

Sur le plan technique, nous avons réussi à concevoir une architecture matérielle fonctionnelle, validée par un bilan de puissance cohérent assurant près de 5 heures d'autonomie théorique. La séparation des circuits de puissance et de logique a garanti la stabilité du Raspberry Pi.

Côté logiciel, l'implémentation du cycle en V nous a permis de structurer notre code efficacement. Une attention particulière a été portée à l'optimisation des tâches : l'utilisation du multithreading pour la gestion des périphériques (buzzer, écran LCD et capteur RGB) a permis de fluidifier la boucle de navigation principale, tandis que l'algorithme de recherche en largeur (BFS) s'est révélé extrêmement performant, calculant le trajet optimal en seulement 60 millisecondes.

La démonstration finale a mis en lumière à la fois la fiabilité de notre logique et les défis du monde physique. Bien que le robot ait parfaitement négocié les 15 premières cases, il a rencontré une difficulté technique liée à la détection des lignes sous les pastilles de couleur. Cet incident montre les limites du code face aux aléas de l'environnement réel.

Au-delà de l'aspect technique, ce projet a été une expérience formatrice en gestion de projet. La coordination entre les pôles électronique et informatique, ainsi que l'utilisation d'outils collaboratifs comme Git, ont été enrichissantes.

Le tableau ci-dessous détaille la répartition des tâches et les contributions de chaque membre de l'équipe ayant permis la réalisation de ce robot. Il est important de préciser que le nombre de commits de chacun n'est pas vraiment représentatif du travail fourni.

Membres	Rôles & Contributions
Assi Hadi Doubet Edouard	Conception électronique, câblage, soudure, perçage du châssis, tests des composants et développement programmes composants
(El Marzouki Adam)	Gestion de projet, organisation, planning
Eraud-Robert Bréhan Antunes Célia	Développement informatique (Algo), structures de données (TAD), analyse descendante et Direction Artistique
Leprevost Romain	Développement informatique (robot), test des composants et ajustements

TABLE 4.1 – Répartition des tâches au sein de l'équipe

4.4 Utilisation de l'IA / de LLM

Dans une démarche de transparence sur l'utilisation de ces outils, nous avons documenté dans un tableau individuel dans quel cadre ces derniers ont été utilisés.

Membres	Rôles & Contributions
Assi Hadi	Questions concernant le branchement du bus I^2C, en particulier les adresses les plus courantes de l'écran LCD et du capteur RGB, le fonctionnement du capteur RGB, ainsi que la définition de plages de valeurs RGB adaptées pour différencier les pastilles du plateau.
Doubet Edouard	Questions concernant les branchements sur l'I2C et les adresses les plus courantes pour l'écran LCD et le capteur RGB, questions sur le fonctionnement du capteur RGB et idées sur des valeurs RGB pour différencier les pastilles du plateau
(El Marzouki Adam)	N/A
Eraud-Robert Bréhan	Base du script bash, support à la génération des tests et de la documentation. Correction de bugs de conception sur le BFS, uniformisation des noms de variables, génération de labyrinthes de test.
Antunes Célia	Renseignement pour les différents algorithmes qui existent pour le plus court chemin et le plus adapté. Correction de bugs et support à la génération des tests. Aide lors d'erreur de compilation inhabituelle
Leprevost Romain	Idées de disposition capteurs / correction de problèmes liés au robot, flowchart rapport, makefile. Tentative sur le code du robot mais très insatisfaisant, besoin de connaître le robot et ses spécificités

TABLE 4.2 – Documentation de l'utilisation de l'IA par membre